

04 アーキテクチャ

04. アーキテクチャ設計（ドラフト）

- ドキュメント種別: 上流工程 / アーキテクチャ設計（物理スキーマ・API・実装方式の確定）
- 対象システム（仮称）: 居酒屋店舗システム（SaaS型） / AIネイティブ再構築版
- 作成日: 2026-09-05
- ステータス: ドラフト（レビュー用）
- 関連文書: 01_system_overview.md、02_requirements.md、03_domain_model.md（本書は 03 第7章の未決事項12件の解決と、物理スキーマ・API・実装方式の確定を行う）

本書は 03_domain_model.md が「04 で確定する」とした論点（物理テーブル定義、テナント分離実装、決済連携詳細、domain_event 実装方式、Mermaid図のPDFレンダリング方針、データ保持期間）と、03 第7章の未決事項12件について、実装に着手できる粒度まで決定する。経営判断・契約・法務確認が必要な項目（02 第11章）はここでも「未確定」として残す。

1. 決定事項一覧（03 第7章 未決事項の解決）

#	論点（03 7章）	決定	詳細
1	company と既存 users の結合方法	company テーブルを新設し、company.id（サロゲートキー）を主キーとする。他の全FK（store_id → store.id 等）と一貫性を持たせ、store・users・user_invitation に company_id BIGINT REFERENCES company(id) を実FKとして追加する。company 自身が company_code（UK）を持つため、この3テーブルは company_code 列を一切持たない（users.company_code も削除し、複合UKを company_id + email に変更する）。それ以外の業務テーブルの company_code 列は非正規化コピー（FK制約なし、_id と名付けない）のまま維持する。	§ 3.1、§ 4
2	卓の結合・分割	フェーズ1は table_session_table による複数卓の結合（占有）のみ実装する。分割（会計途中で order_line を別セッションへ移送）はフェーズ2へ送る（03 の区分どおり § → フェーズ2）。	§ 4
3	オフライン同期の競合解決規則	クライアント一時IDは「端末ID（サーバがデバイス登録時に発番。store 短縮コード+店内連番）+端末内でグローバルに単調増加する連番」の合成文字列を、customer_order・order_line・order_line_option の各レコードの client_ref_id（VARCHAR(40)）に持たせ、各テーブルの UNIQUE(..., client_ref_id) と staff_device.last_accepted_seq でサブツリー全階層の冪等性を担保する。同期ペイロードはフラットなレコード配列+FK列に一時IDの形式とし、作り直し参照は remake_of_line_id（確定ID）と remake_of_line_ref（一時ID、ペイロード専用）に分けて持つ。バッチ内の依存順はアプリ層のトポロジカルソートで解決する（DBの遅延制約は使わない）。同期レスポンスは、エンベロープ（server_received_at / last_accepted_seq）+送信レコード全件に対応するフラットな records[]（type / client_ref_id / id / status / 任意の server_fields・error）を返す。クライアント側の一時ID⇔確定ID対応表は、対象 table_session が CLOSED になるまで保持し以後破棄する（バックストップの固定TTL付き）。部分失敗はハイブリッド（サブツリー前提の検証はall-or-nothing、個々の明細検証は部分コミット）で扱い、落ちた明細は REJECTED + error.code、その子は SKIPPED で返	§ 9

		す。error.code 付き REJECTED は永続的失敗（自動再送せずスタッフヘエスケーレーション）とし、一時的失敗は 5xx/タイムアウト等で判定してバッチ全体を指数バックオフ再送する。端末の再セットアップ時は端末IDを再発番し、旧IDは再利用しない。オフライン中は注文明細の新規追加のみ許可し、数量変更・取消・会計・決済はオンライン復帰後にのみ許可することで、更新競合そのものを設計上発生させない。永続化済み行への更新は serve_status の PENDING→SERVED を含めオフライン不可とし、updated_at / version 比較の楽観的ロックでの解禁も採用しない（既存行の更新意図はローカルキューへ退避し復帰後にオンライン操作として再生）。遅延同期が CLOSED セッション / FINALIZED check に着地した場合はフェーズ1では代金回収せず、回収不能（廃棄ロス/サービス提供分）として domain_event に記録するのみとする（オフライン許容時間の上限は未決）。オフライン作成明細のメニュースナップショット（名称・価格・税区分）は端末保持のキャッシュ値で確定し、サーバは復帰時に再価格付けしない。オフライン作成レコードの時刻は端末時計+スキュー補正（リクエストに client_sent_at を追加し offset = server_received_at - client_sent_at をバッチ内の各時刻へ一律加算）で確定し、補正値は低信頼フラグ付きで格納、時系列的に破綻する時刻のみ server_received_at へ置換する。order_line に business_date 列を追加し、補正後 registered_at +店舗の営業日境界から算出（低信頼明細は guest_check.business_date を継承）。算出先が締め済み営業日なら拒否せずオープン中の営業日へ寄せて理由コード付きで当日計上（D2、daily_close は不変のまま）。offset 許容上限 X はシステム全体の運用設定値（店舗別オーバーライドはフェーズ2以降）とし、格納方式・既定値・レンジは実装スパイクで確定。KDS のチケット表示順は printed_at ではなく注文入力時刻（submitted_at / registered_at、オフラインは補正後）を基準にする。オフライン作成明細はペイロードに serve_status / served_at を初期状態として載せ（案1）、kitchen_ticket は customer_order 単位のまま、全明細 SERVED のオーダーのみ status = DONE ・KDS 非表示、PENDING を含むオーダーはKDS に出し表示明細を PENDING に絞る（G1）。ミュート照合レーンの要否・遅延計上/低信頼フラグの列名は未決。	
4	domain_event の粒度・保持方針	集約単位（TABLE_SESSION / ORDER_LINE / CHECK / PAYMENT / DAILY_CLOSE）の主要状態変化のみを記録（列変更の逐一記録はしない）。直近13か月はオンラインテーブル、それ以降は月次パーティションでコールドストレージへ退避し、10年で削除する。	§ 8
5	モバイルオーダー qr_token の設計	2層構成を採用。dining_table.qr_token は卓に固定された長命トークン（店舗設定画面から手動再発行可）。mobile_order_session.qr_token は読み取りの都度発行される短命トークンで、卓クローズ時に失効する。	§ 7.4
6	税計算の丸め・端数調整	インボイス制度の要求に従い、1会計（適格請求書）につき税率区分ごとに1回だけ端数処理する（check_tax_line 単位）。端数処理方式は切り捨てを既定とする。現金精算等で生じる1円未満の調整は check_discount.type = ROUNDING で表現する。	§ 6.4
7	receipt の生成方式	サーバ生成。外部バイナリ依存を避けるため openhtmltopdf（Java純正のHTML→PDFライブラリ）をバックエンドに組み込み、Thymeleafテンプレートから生成する。ドキュメントPDF化（本リポジトリの docs/pdf/）で用いた wkhtmltopdf 方式は開発ドキュメント用途に限り、本番の帳票生成には採用しない。	§ 6.5
8	staff と user の一体化度合い	03 の設計（staff.user_id nullable、user と 0..1:1）のまま確定。打刻のみ行う非ログインスタッフ（user_id IS NULL）を許容する。	§ 4
9	売上日報の「客数」「組数」の定義	客数 = 当日クローズした table_session.party_size の合計。組数 = 当日クローズした table_session の件数。	§ 4

10	menu_option_group / menu_option / course のフェーズ1採用可否	フェーズ1に含める（居酒屋業態でコース・飲み放題・トッピングは頻出機能であり、実装コストがテーブル追加のみで小さいため）。	§ 4
11	（フェーズ2構想）guest エンティティ	本書では物理設計を行わない。03 未決事項11の方針（user とは別の認証経路、reservation.guest_id nullable 追加）を踏襲し、フェーズ2着手時に本書を改訂する。	—
12	Mermaid図のPDFレンダリング方針	pandoc --pdf-engine=wkhtmltopdf を採用し、mermaid フェンスは事前に @mermaid-js/mermaid-cli（npm @mermaid-js/mermaid-cli）でPNG化してから埋め込む。2026-09-05に本リポジトリの docs/pdf/*.pdf で運用実績あり。	§ 12

2. 全体アーキテクチャ（確定）

01 第5章の構成案を、以下のとおり確定する（変更点のみ記載。それ以外は 01 5.2～5.6 のとおり）。

- フロントエンド：React 19 + TypeScript + Vite。admin / pos / guest の3アプリ構成。
- バックエンド：Spring Boot 4.x / Java 21 / Maven。パッケージルートは既存踏襲で com.shopsystem.backend。
- DB：PostgreSQL。マイグレーション管理は **Flyway** を採用する（src/main/resources/db/migration/V<n>__<desc>.sql）。Hibernate の ddl-auto はフェーズ1から validate 固定とし、スキーマ変更は必ずマイグレーションファイル経由で行う。
- AI/LLM 基盤：フェーズ1では実装しない（01 5.5 のとおり差し込み位置だけ確保）。

3. テナント分離の実装方式

3.1 データモデル上の分離

- company を新設し、id BIGINT（サロゲートキー）を主キーとする。company_code（varchar(20)）は登録・ログイン画面で人間が入力する自然キーとして UNIQUE NOT NULL を維持するが、他テーブルからの参照キー（FK）としては使わない。
- store・users・user_invitation は company に直接ぶら下がる最上位のテーブルであるため、他の全FK（store_id → store.id、category_id → menu_category.id 等）と一貫性を持たせ、company_id BIGINT NOT NULL REFERENCES company(id) を実FKとして持つ。company 自身が company_code（UK）を持つため、この3テーブルに company_code 列は一切持たせない。既存 users.company_code は本書のマイグレーションで company_id の追加・バックフィル後に削除する（調査の結果、現時点でログイン機能は未実装〈登録APIのみ〉であり、本番で company_code に依存する稼働中の認証フローは存在しないため、後方互換のために残す理由はない）。company_id 追加前に、既存データに存在する company_code の重複しない値ごとに company 行をバックフィルする。
- users の複合ユニーク制約は company_code + email から company_id + email に変更する（uk_users_company_code_email を uk_users_company_id_email に置き換え）。登録・ログイン画面で利用者が入力する company_code は、リクエスト処理時に company テーブルを WHERE company_code = ? で検索して company_id に変換してから使う（DTO・リクエストボディの入力項目としては残るが、users テーブルの列としては持たない）。
- store 配下の全業務テーブルは store_id BIGINT NOT NULL REFERENCES store(id) を持つ（user.store_id のみ「全店」を表す NULL を許容）。store_id が既に store.company_id を経由して会社を一意に特定できるため、store・users・user_invitation 以外の業務テーブルには company_id を追加しない。
- それ以外の業務テーブル（menu_category、reservation、table_session、audit_log、domain_event 等）が持つ company_code 列は、store_id から company まで複数ホップの結合を経ずにテナント単位の集計・インデックスを可能にするための非正規化列であり、アプリ層が company_id（または store_id 経由）から引いた company.company_code を書き込む。DBレベルのFK制約は張らない（整合性は store_id → store.company_id → company の連鎖とアプリ層のテナント強制〈§ 3.2〉で担保する）。この列をあえて company_id と名付けない理由は § 4.1 の命名規約を参照（_id は実FK専用の命名）。

3.2 アプリ層での強制（フェーズ1で採用する方式）

- 認証済みリクエストのコンテキスト（JWT のクレーム）から `company_id` / `company_code`、ユーザーの `store_id`（`NULL` = 全店）を解決し、リクエストスコープの `TenantContext`（`ThreadLocal` ベース）に保持する。
- 全リポジトリは `BaseRepository<T>` を継承し、`findById` 系を含むすべての参照・更新メソッドでテナント条件を `WHERE` に強制注入する。Spring Data JPA では Hibernate の `@FilterDef` / `@Filter` を2種類定義する：`company_code` 列を持つエンティティ（`menu_item`、`reservation` 等の業務テーブル）には `company_code = :tenantCompanyCode`、`company_id` 列を持つエンティティ（`Store`、`User`、`UserInvitation`）には `company_id = :tenantCompanyId` を適用する。いずれも `OpenEntityManagerInViewInterceptor` 相当のフィルタでリクエスト開始時に有効化する。
- サービス層でテナントを跨ぐクエリを書けないよう、`company_code` を引数に取らないカスタムクエリの追加を禁止するアーキテクチャテスト（`ArchUnit`）をCIに組み込む。

3.3 Row Level Security（RLS）の要否 → フェーズ1では採用しない

- 理由：共有DB・共有スキーマ+アプリ層強制（Hibernate Filter+ArchUnit静的検査）の二重の防御で、フェーズ1のリスク（1テナント1店舗、社内の少人数運用）に対しては十分と判断。RLS はロールごとの `SET app.current_company_code` の運用（コネクションプーリング時のセッション変数管理）が煩雑で、少人数運用（NFR-14）の方針と相性が悪い。
- 将来の見直し条件：大口テナントの専用DB分離、または監査要件の強化（例：SOC2 対応）が必要になった時点で、RLS もしくはテナントごとのスキーマ分離への移行を検討する（NFR-07 の設計余地どおり）。

4. 物理スキーマ（DDL）

4.1 命名・型の規約（03 6章を継承・確定）

- テーブル名・カラム名は `snake_case`、単数形。ただし **SQL予約語と衝突する物理名は変更する**：
 - ドメイン概念「注文（`order`）」の物理テーブル名は `customer_order` とする（`ORDER BY` との衝突回避）。
 - ドメイン概念「会計（`check`）」の物理テーブル名は `guest_check` とする（`CHECK` 制約キーワードとの衝突回避）。これに伴い
`check_line` → `guest_check_line`、`check_discount` → `guest_check_discount`、
`check_tax_line` → `guest_check_tax_line` とする。
 - アプリケーション層（エンティティクラス名・API・ドメイン用語）は 03 のとおり `Order` / `Check` を使い続けてよい。物理テーブル名との対応は ORM のマッピングで吸収する。
- 主キー：`id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`（JPA `GenerationType.IDENTITY` 互換）。以下のDDLでは簡潔さのため `BIGINT PK` と表記する。
- 外部キーは `<entity>_id BIGINT REFERENCES <table>(id)`。以下のDDLでは `BIGINT FK -> <table>` と表記する。**`_id` サフィックスは実際にFK制約が張られた列にのみ使う**。`company_code` のようにFK制約を持たない非正規化コピー列（§3.1）は、`_id` を使わず元の自然キーの名前のまま保持し、実FKと区別できるようにする。
- 金額は `INTEGER`（円、小数なし）、単位サフィックス `_jpy`。時刻は `TIMESTAMPZ`（列名 `_at`）。日付は `DATE`（列名 `_date` または `business_date`）。時刻のみは `TIME`。
- 列挙（enum）は `VARCHAR(30)` 文字列で保持し、`CHECK (col IN (...))` 制約で許容値を固定する（DBネイティブ `ENUM` 型は使わない。値追加時のマイグレーションを軽くするため）。
- 監査列：全テーブルに `created_at TIMESTAMPZ NOT NULL DEFAULT now()`、`updated_at TIMESTAMPZ`。業務テーブルにはさらに `created_by VARCHAR(255) NOT NULL`、`updated_by VARCHAR(255)`（既存 `BaseEntity` を踏襲。値はユーザーのメールアドレスまたは `SYSTEM`）。以下のDDLでは省略し、末尾に共通定義として一括で示す。
- 論理削除：マスタ系テーブルは `is_active BOOLEAN NOT NULL DEFAULT true` を持ち、物理削除しない。

4.2 共通監査列（全業務テーブルに付与）

```
-- すべての CREATE TABLE の列リストの末尾に、以下を付与する（本書のDDLでは記載を省略）
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
created_by VARCHAR(255) NOT NULL,
updated_at TIMESTAMPTZ,
updated_by VARCHAR(255)
```

4.3 テナント・組織・ユーザー

```
CREATE TABLE company (
  id BIGINT PK,
  company_code VARCHAR(20) NOT NULL UNIQUE,
  name VARCHAR(255) NOT NULL,
  contract_status VARCHAR(30) NOT NULL DEFAULT 'ACTIVE'
  CHECK (contract_status IN ('ACTIVE', 'SUSPENDED', 'CANCELLED'))
);

CREATE TABLE store (
  id BIGINT PK,
  company_id BIGINT NOT NULL REFERENCES company(id),
  name VARCHAR(255) NOT NULL,
  address VARCHAR(500),
  phone VARCHAR(30),
  business_hours VARCHAR(255),
  seat_count INTEGER NOT NULL DEFAULT 0,
  timezone VARCHAR(50) NOT NULL DEFAULT 'Asia/Tokyo',
  is_active BOOLEAN NOT NULL DEFAULT true
);
CREATE INDEX ix_store_company_id ON store(company_id);
-- store は company から1ホップのため company_code の非正規化コピーは持たない (§3.1)。

CREATE TABLE store_setting (
  store_id BIGINT PK REFERENCES store(id),
  tax_rounding VARCHAR(20) NOT NULL DEFAULT 'FLOOR'
  CHECK (tax_rounding IN ('FLOOR', 'CEIL', 'ROUND')),
  price_includes_tax BOOLEAN NOT NULL DEFAULT true,
  invoice_reg_no VARCHAR(20),
  web_reservation_mode VARCHAR(20) NOT NULL DEFAULT 'APPROVAL'
  CHECK (web_reservation_mode IN ('APPROVAL', 'INSTANT')),
  mobile_order_enabled BOOLEAN NOT NULL DEFAULT false,
  last_order_default_min INTEGER NOT NULL DEFAULT 30,
  cancel_charge_default_customer BOOLEAN NOT NULL DEFAULT true,
  cancel_charge_default_store BOOLEAN NOT NULL DEFAULT false
);

-- 既存 users テーブルへのマイグレーション（company を新設した後に適用）
ALTER TABLE users
  ADD COLUMN company_id BIGINT REFERENCES company(id), -- バックフィル後に NOT NULL 化
  ADD COLUMN store_id BIGINT REFERENCES store(id),
  ADD COLUMN role VARCHAR(20) NOT NULL DEFAULT 'OWNER'
  CHECK (role IN ('OWNER', 'MANAGER', 'HALL', 'KITCHEN', 'PARTTIME')),
  ADD COLUMN status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'
  CHECK (status IN ('ACTIVE', 'LOCKED', 'INVITED')),
  ADD COLUMN two_factor_enabled BOOLEAN NOT NULL DEFAULT false;
-- バックフィル（company_code ごとに対応する company.id を引いて users.company_id に設定）後、以下を実行する
-- ALTER TABLE users ALTER COLUMN company_id SET NOT NULL;
-- ALTER TABLE users DROP CONSTRAINT uk_users_company_code_email;
-- ALTER TABLE users ADD CONSTRAINT uk_users_company_id_email UNIQUE (company_id, email);
-- ALTER TABLE users DROP COLUMN company_code; -- company に company_code(UK) があるため users には持たせない
-- users.telnumber は既存カラムをそのまま流用。

CREATE TABLE user_invitation (
```

```

    id          BIGINT PK,
    company_id  BIGINT NOT NULL REFERENCES company(id), -- 低頻度の管理操作のため非正規化せず実FKにする
    store_id    BIGINT REFERENCES store(id),
    email       VARCHAR(255) NOT NULL,
    role        VARCHAR(20) NOT NULL
               CHECK (role IN ('OWNER', 'MANAGER', 'HALL', 'KITCHEN', 'PARTTIME')),
    token       VARCHAR(255) NOT NULL UNIQUE,
    expires_at  TIMESTAMPTZ NOT NULL,
    accepted_at TIMESTAMPTZ
);
CREATE INDEX ix_user_invitation_token ON user_invitation(token);

```

4.4 マスタ（メニュー・卓・コース・決済手段）

```

CREATE TABLE menu_category (
    id          BIGINT PK,
    company_code VARCHAR(20) NOT NULL, -- 非正規化コピー（FK制約なし。§3.1）
    store_id    BIGINT NOT NULL REFERENCES store(id),
    name        VARCHAR(100) NOT NULL,
    display_order INTEGER NOT NULL DEFAULT 0,
    is_active   BOOLEAN NOT NULL DEFAULT true
);

CREATE TABLE menu_item (
    id          BIGINT PK,
    company_code VARCHAR(20) NOT NULL, -- 非正規化コピー（FK制約なし。§3.1）
    store_id    BIGINT NOT NULL REFERENCES store(id),
    category_id BIGINT NOT NULL REFERENCES menu_category(id),
    name        VARCHAR(255) NOT NULL,
    description  TEXT,
    price_jpy   INTEGER NOT NULL,
    tax_category VARCHAR(20) NOT NULL CHECK (tax_category IN ('STANDARD_10', 'REDUCED_8')),
    photo_url   VARCHAR(500),
    serve_time_from TIME,
    serve_time_to   TIME,
    available_from  TIMESTAMPTZ,
    available_to    TIMESTAMPTZ,
    sales_status    VARCHAR(20) NOT NULL DEFAULT 'ON_SALE'
               CHECK (sales_status IN ('ON_SALE', 'SOLD_OUT', 'SUSPENDED')),
    display_order   INTEGER NOT NULL DEFAULT 0,
    is_active       BOOLEAN NOT NULL DEFAULT true
);
CREATE INDEX ix_menu_item_store_category ON menu_item(store_id, category_id);

CREATE TABLE menu_option_group (
    id          BIGINT PK,
    store_id    BIGINT NOT NULL REFERENCES store(id),
    menu_item_id BIGINT REFERENCES menu_item(id),
    name        VARCHAR(100) NOT NULL,
    min_select  INTEGER NOT NULL DEFAULT 0,
    max_select  INTEGER NOT NULL DEFAULT 1
);

CREATE TABLE menu_option (
    id          BIGINT PK,
    option_group_id BIGINT NOT NULL REFERENCES menu_option_group(id),
    name        VARCHAR(100) NOT NULL,
    price_delta_jpy INTEGER NOT NULL DEFAULT 0,
    is_active   BOOLEAN NOT NULL DEFAULT true
);

```

```

CREATE TABLE course (
  id          BIGINT PK,
  store_id    BIGINT NOT NULL REFERENCES store(id),
  name        VARCHAR(255) NOT NULL,
  type        VARCHAR(20) NOT NULL CHECK (type IN ('COURSE', 'FREE_DRINK')),
  duration_min INTEGER NOT NULL,
  last_order_before_min INTEGER NOT NULL DEFAULT 30,
  price_jpy   INTEGER NOT NULL,
  is_active   BOOLEAN NOT NULL DEFAULT true
);

CREATE TABLE dining_table (
  id          BIGINT PK,
  store_id    BIGINT NOT NULL REFERENCES store(id),
  table_no    VARCHAR(20) NOT NULL,
  seat_count  INTEGER NOT NULL,
  area        VARCHAR(100),
  qr_token    VARCHAR(64) NOT NULL,
  status      VARCHAR(20) NOT NULL DEFAULT 'EMPTY'
  CHECK (status IN ('EMPTY', 'OCCUPIED', 'BILLING')),
  is_active   BOOLEAN NOT NULL DEFAULT true,
  UNIQUE (store_id, table_no),
  UNIQUE (store_id, qr_token)
);

CREATE TABLE payment_method_config (
  id          BIGINT PK,
  store_id    BIGINT NOT NULL REFERENCES store(id),
  method_type VARCHAR(20) NOT NULL
  CHECK (method_type IN ('CASH', 'PAYPAY', 'CREDIT_CARD', 'RAKUTEN_PAY')),
  enabled     BOOLEAN NOT NULL DEFAULT false,
  display_name VARCHAR(100),
  credential_enc BYTEA,
  note        VARCHAR(500),
  UNIQUE (store_id, method_type)
);

CREATE TABLE store_business_day (
  id          BIGINT PK,
  store_id    BIGINT NOT NULL REFERENCES store(id),
  business_date DATE,
  weekday     SMALLINT CHECK (weekday BETWEEN 0 AND 6),
  is_open     BOOLEAN NOT NULL DEFAULT true,
  open_time   TIME,
  close_time  TIME,
  reservation_capacity INTEGER,
  CHECK ((business_date IS NOT NULL) <> (weekday IS NOT NULL))
);

```

4.5 予約

```

CREATE TABLE reservation (
  id          BIGINT PK,
  company_code VARCHAR(20) NOT NULL, -- 非正規化コピー (FK制約なし。S3.1)
  store_id    BIGINT NOT NULL REFERENCES store(id),
  reserved_at TIMESTAMPTZ NOT NULL,
  party_size  INTEGER NOT NULL,
  guest_name  VARCHAR(100) NOT NULL,
  guest_phone VARCHAR(30),
  guest_email VARCHAR(255),

```

```

request_note    TEXT,
course_id       BIGINT REFERENCES course(id),
channel         VARCHAR(20) NOT NULL CHECK (channel IN ('WEB', 'PHONE', 'WALK_IN')),
status          VARCHAR(20) NOT NULL DEFAULT 'REQUESTED'
    CHECK (status IN ('REQUESTED', 'CONFIRMED', 'SEATED', 'DONE', 'CANCELLED', 'NO_SHOW')),
confirmed_by    VARCHAR(255),
confirmed_at    TIMESTAMPTZ,
cancelled_reason VARCHAR(255)
);
CREATE INDEX ix_reservation_store_date ON reservation(store_id, reserved_at);

CREATE TABLE reservation_notification (
    id            BIGINT PK,
    reservation_id BIGINT NOT NULL REFERENCES reservation(id),
    type          VARCHAR(20) NOT NULL CHECK (type IN ('CONFIRM', 'REMINDER')),
    channel       VARCHAR(20) NOT NULL CHECK (channel IN ('EMAIL', 'SMS')),
    "to"          VARCHAR(255) NOT NULL,
    status        VARCHAR(20) NOT NULL DEFAULT 'QUEUED'
    CHECK (status IN ('QUEUED', 'SENT', 'FAILED')),
    sent_at       TIMESTAMPTZ
);

```

4.6 来店・卓セッション・注文

```

CREATE TABLE table_session (
    id            BIGINT PK,
    company_code  VARCHAR(20) NOT NULL, -- 非正規化コピー (FK制約なし。§3.1)
    store_id      BIGINT NOT NULL REFERENCES store(id),
    status        VARCHAR(20) NOT NULL DEFAULT 'OPEN'
    CHECK (status IN ('OPEN', 'BILLING', 'CLOSED')),
    opened_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    opened_by     VARCHAR(255) NOT NULL,
    closed_at     TIMESTAMPTZ,
    party_size    INTEGER NOT NULL,
    reservation_id BIGINT REFERENCES reservation(id),
    course_id     BIGINT REFERENCES course(id),
    course_started_at TIMESTAMPTZ,
    last_order_at TIMESTAMPTZ
);
CREATE INDEX ix_table_session_store_status ON table_session(store_id, status);

CREATE TABLE table_session_table (
    table_session_id BIGINT NOT NULL REFERENCES table_session(id),
    dining_table_id  BIGINT NOT NULL REFERENCES dining_table(id),
    is_primary       BOOLEAN NOT NULL DEFAULT true,
    PRIMARY KEY (table_session_id, dining_table_id)
);

CREATE TABLE mobile_order_session (
    id            BIGINT PK,
    table_session_id BIGINT NOT NULL REFERENCES table_session(id),
    qr_token      VARCHAR(64) NOT NULL UNIQUE,
    issued_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at    TIMESTAMPTZ NOT NULL,
    status        VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'
    CHECK (status IN ('ACTIVE', 'EXPIRED'))
);

CREATE TABLE staff_device (
    id            BIGINT PK,
    company_code  VARCHAR(20) NOT NULL, -- 非正規化コピー (FK制約なし。§3.1)

```

```

store_id          BIGINT NOT NULL REFERENCES store(id),
device_code       VARCHAR(20) NOT NULL, -- 端末ID。store短縮コード + 店内連番。例 'S12-07'
label            VARCHAR(100),          -- 表示名 ('ホール1号機' 等)
status            VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'
                CHECK (status IN ('ACTIVE', 'RETIRED')), -- 再セットアップ時は RETIRED にし新 device_code を再発番 (li
コードは再利用しない)
last_accepted_seq BIGINT NOT NULL DEFAULT 0, -- この端末から受理済みの最大連番。これ以下の連番を持つ注3
再送とみなす
retired_at        TIMESTAMPTZ,
UNIQUE (store_id, device_code)
);

CREATE TABLE customer_order (
    id              BIGINT PK,
table_session_id  BIGINT NOT NULL REFERENCES table_session(id),
source            VARCHAR(20) NOT NULL CHECK (source IN ('STAFF', 'MOBILE')),
entered_by        VARCHAR(255),
mobile_order_session_id BIGINT REFERENCES mobile_order_session(id),
client_ref_id     VARCHAR(40), -- 端末ID + "-" + 端末内グローバル連番。例 'S12-07-000123' (§9)
status            VARCHAR(20) NOT NULL DEFAULT 'SUBMITTED'
                CHECK (status IN ('SUBMITTED', 'ACCEPTED', 'REJECTED')),
submitted_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
accepted_by       VARCHAR(255),
accepted_at       TIMESTAMPTZ,
reject_reason     VARCHAR(255),
UNIQUE (table_session_id, client_ref_id)
);

CREATE TABLE order_line (
    id              BIGINT PK,
order_id          BIGINT NOT NULL REFERENCES customer_order(id),
table_session_id  BIGINT NOT NULL REFERENCES table_session(id),
menu_item_id      BIGINT NOT NULL REFERENCES menu_item(id),
item_name_snap    VARCHAR(255) NOT NULL,
unit_price_snap_jpy INTEGER NOT NULL,
tax_category_snap VARCHAR(20) NOT NULL,
quantity          INTEGER NOT NULL,
note              VARCHAR(500),
serve_status      VARCHAR(20) NOT NULL DEFAULT 'PENDING'
                CHECK (serve_status IN ('PENDING', 'PREPARING', 'SERVED', 'CANCELLED', 'REJECTED')),
registered_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
registered_by     VARCHAR(255) NOT NULL,
business_date     DATE NOT NULL, -- 注文された営業日。登録時に registered_at (オフラインは補正後) へ
舗の営業日境界から算出 (§9)
time_low_confidence BOOLEAN NOT NULL DEFAULT false, -- (仮称) 端末時計のスキュー補正が X 超過/時系列
でサーバ時刻置換のとき true (§9)
served_at         TIMESTAMPTZ,
cancelled_at       TIMESTAMPTZ,
cancelled_by       VARCHAR(255),
cancel_reason     VARCHAR(20)
                CHECK (cancel_reason IN
('ORDER_MISTAKE', 'QUALITY', 'DELAY', 'WRONG_SERVE', 'SOLD_OUT', 'CUSTOMER', 'OTHER')),
cancel_chargeable BOOLEAN,
was_cooked         BOOLEAN,
remake_of_line_id BIGINT REFERENCES order_line(id),
client_ref_id     VARCHAR(40), -- 端末ID + "-" + 端末内グローバル連番。オフライン作成分のみ (§9)
UNIQUE (table_session_id, client_ref_id)
);
CREATE INDEX ix_order_line_table_session ON order_line(table_session_id);
CREATE INDEX ix_order_line_business_date ON order_line(business_date); -- 明細粒度の日次集計・分析用 (§9)

CREATE TABLE order_line_option (

```

```

    id                BIGINT PK,
    order_line_id     BIGINT NOT NULL REFERENCES order_line(id),
    option_name_snap  VARCHAR(100) NOT NULL,
    price_delta_snap_jpy INTEGER NOT NULL DEFAULT 0,
    client_ref_id     VARCHAR(40), -- オフライン作成成分のみ (S9)
    UNIQUE (order_line_id, client_ref_id)
);

CREATE TABLE kitchen_ticket (
    id                BIGINT PK,
    order_id          BIGINT NOT NULL REFERENCES customer_order(id),
    store_id          BIGINT NOT NULL REFERENCES store(id),
    status            VARCHAR(20) NOT NULL DEFAULT 'NEW'
    CHECK (status IN ('NEW', 'IN_PROGRESS', 'DONE')),
    printed_at        TIMESTAMPTZ
);

```

4.7 会計・決済

```

CREATE TABLE guest_check (
    id                BIGINT PK,
    table_session_id  BIGINT NOT NULL REFERENCES table_session(id),
    store_id          BIGINT NOT NULL REFERENCES store(id),
    seq_in_session    INTEGER NOT NULL,
    status            VARCHAR(20) NOT NULL DEFAULT 'OPEN'
    CHECK (status IN ('OPEN', 'FINALIZED', 'VOIDED')),
    subtotal_jpy      INTEGER NOT NULL DEFAULT 0,
    discount_total_jpy INTEGER NOT NULL DEFAULT 0,
    tax_total_jpy     INTEGER NOT NULL DEFAULT 0,
    total_jpy         INTEGER NOT NULL DEFAULT 0,
    split_type        VARCHAR(20) NOT NULL DEFAULT 'NONE'
    CHECK (split_type IN ('NONE', 'BY_HEAD', 'BY_ITEM')),
    split_count       INTEGER,
    business_date      DATE NOT NULL,
    finalized_at       TIMESTAMPTZ,
    finalized_by       VARCHAR(255),
    UNIQUE (table_session_id, seq_in_session)
);

CREATE INDEX ix_guest_check_store_business_date ON guest_check(store_id, business_date);

CREATE TABLE guest_check_line (
    id                BIGINT PK,
    check_id          BIGINT NOT NULL REFERENCES guest_check(id),
    order_line_id     BIGINT NOT NULL REFERENCES order_line(id),
    amount_jpy        INTEGER NOT NULL,
    quantity          INTEGER NOT NULL,
    UNIQUE (order_line_id) -- 1つの order_line は1つの guest_check にのみ割当 (S5 不変条件12)
);

CREATE TABLE guest_check_discount (
    id                BIGINT PK,
    check_id          BIGINT NOT NULL REFERENCES guest_check(id),
    type              VARCHAR(20) NOT NULL CHECK (type IN ('AMOUNT', 'RATE', 'COUPON', 'ROUNDING')),
    value             NUMERIC(10,2) NOT NULL,
    amount_jpy        INTEGER NOT NULL,
    reason            VARCHAR(255),
    applied_by        VARCHAR(255) NOT NULL,
    applied_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE guest_check_tax_line (

```

```

    id          BIGINT PK,
    check_id    BIGINT NOT NULL REFERENCES guest_check(id),
    tax_category VARCHAR(20) NOT NULL CHECK (tax_category IN ('STANDARD_10', 'REDUCED_8')),
    taxable_amount_jpy INTEGER NOT NULL,
    tax_amount_jpy INTEGER NOT NULL,
    UNIQUE (check_id, tax_category)
);

CREATE TABLE payment (
    id          BIGINT PK,
    check_id    BIGINT NOT NULL REFERENCES guest_check(id),
    method_type VARCHAR(20) NOT NULL
    CHECK (method_type IN ('CASH', 'PAYPAY', 'CREDIT_CARD', 'RAKUTEN_PAY')),
    amount_jpy  INTEGER NOT NULL,
    status      VARCHAR(20) NOT NULL DEFAULT 'PENDING'
    CHECK (status IN ('SUCCESS', 'FAILED', 'PENDING')),
    external_txn_id VARCHAR(100),
    tendered_jpy  INTEGER,
    change_jpy    INTEGER,
    is_manual_entry BOOLEAN NOT NULL DEFAULT false,
    processed_at  TIMESTAMPTZ,
    processed_by  VARCHAR(255) NOT NULL
);

CREATE UNIQUE INDEX ux_payment_external_txn ON payment(method_type, external_txn_id)
WHERE external_txn_id IS NOT NULL; -- Webhook再送の幂等性 (§ 7.3)

CREATE TABLE refund (
    id          BIGINT PK,
    check_id    BIGINT NOT NULL REFERENCES guest_check(id),
    payment_id  BIGINT REFERENCES payment(id),
    amount_jpy  INTEGER NOT NULL,
    reason      VARCHAR(255) NOT NULL,
    reason_note VARCHAR(500),
    executed_by VARCHAR(255) NOT NULL,
    executed_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    approved_by VARCHAR(255)
);

CREATE TABLE receipt (
    id          BIGINT PK,
    check_id    BIGINT NOT NULL REFERENCES guest_check(id),
    type        VARCHAR(20) NOT NULL CHECK (type IN ('RECEIPT', 'INVOICE')),
    addressee   VARCHAR(255),
    proviso     VARCHAR(255),
    issued_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    pdf_ref     VARCHAR(500) NOT NULL,
    invoice_reg_no_snap VARCHAR(20),
    tax_lines_snap JSONB NOT NULL
);

```

4.8 日次締め・売上日報

```

CREATE TABLE daily_close (
    id          BIGINT PK,
    store_id    BIGINT NOT NULL REFERENCES store(id),
    business_date DATE NOT NULL,
    status      VARCHAR(20) NOT NULL DEFAULT 'OPEN'
    CHECK (status IN ('OPEN', 'CLOSED')),
    closed_by   VARCHAR(255),
    closed_at   TIMESTAMPTZ,
    cash_counted_jpy INTEGER,

```

```

        cash_theoretical_jpy    INTEGER,
        cash_diff_jpy          INTEGER,
        UNIQUE (store_id, business_date)
    );

    CREATE TABLE sales_daily_report (
        id                      BIGINT PK,
        daily_close_id          BIGINT NOT NULL REFERENCES daily_close(id),
        store_id                 BIGINT NOT NULL REFERENCES store(id),
        business_date            DATE NOT NULL,
        sales_total_jpy          INTEGER NOT NULL,
        guest_count              INTEGER NOT NULL,
        group_count              INTEGER NOT NULL,
        avg_per_guest_jpy        INTEGER NOT NULL,
        discount_total_jpy       INTEGER NOT NULL,
        refund_total_jpy         INTEGER NOT NULL
    );

    CREATE TABLE sales_report_by_payment (
        id                      BIGINT PK,
        sales_daily_report_id    BIGINT NOT NULL REFERENCES sales_daily_report(id),
        method_type              VARCHAR(20) NOT NULL,
        amount_jpy               INTEGER NOT NULL,
        txn_count                 INTEGER NOT NULL
    );

    CREATE TABLE sales_report_by_tax (
        id                      BIGINT PK,
        sales_daily_report_id    BIGINT NOT NULL REFERENCES sales_daily_report(id),
        tax_category              VARCHAR(20) NOT NULL,
        sales_amount_jpy          INTEGER NOT NULL,
        tax_amount_jpy            INTEGER NOT NULL
    );

    CREATE TABLE sales_report_by_hour (
        id                      BIGINT PK,
        sales_daily_report_id    BIGINT NOT NULL REFERENCES sales_daily_report(id),
        hour                     SMALLINT NOT NULL CHECK (hour BETWEEN 0 AND 23),
        sales_amount_jpy          INTEGER NOT NULL,
        guest_count              INTEGER NOT NULL
    );

```

4.9 スタッフ・シフト・勤怠

```

    CREATE TABLE staff (
        id                      BIGINT PK,
        store_id                 BIGINT NOT NULL REFERENCES store(id),
        user_id                  BIGINT REFERENCES users(id),
        name                     VARCHAR(100) NOT NULL,
        role                     VARCHAR(50),
        hourly_wage_jpy          INTEGER,
        contact                   VARCHAR(255),
        is_active                 BOOLEAN NOT NULL DEFAULT true
    );

    CREATE TABLE staff_availability (
        id                      BIGINT PK,
        staff_id                 BIGINT NOT NULL REFERENCES staff(id),
        weekday                  SMALLINT NOT NULL CHECK (weekday BETWEEN 0 AND 6),
        start_time                TIME NOT NULL,
        end_time                  TIME NOT NULL
    );

```

```

);

CREATE TABLE shift_request (
  id          BIGINT PK,
  staff_id    BIGINT NOT NULL REFERENCES staff(id),
  target_date DATE NOT NULL,
  type        VARCHAR(20) NOT NULL CHECK (type IN ('AVAILABLE', 'UNAVAILABLE')),
  start_time  TIME,
  end_time    TIME,
  comment     VARCHAR(500),
  submitted_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE shift_schedule (
  id          BIGINT PK,
  store_id    BIGINT NOT NULL REFERENCES store(id),
  period_start DATE NOT NULL,
  period_end   DATE NOT NULL,
  status       VARCHAR(20) NOT NULL DEFAULT 'DRAFT'
              CHECK (status IN ('DRAFT', 'PUBLISHED')),
  published_at TIMESTAMPTZ,
  created_by   VARCHAR(255) NOT NULL
);

CREATE TABLE shift_assignment (
  id          BIGINT PK,
  shift_schedule_id BIGINT NOT NULL REFERENCES shift_schedule(id),
  staff_id    BIGINT NOT NULL REFERENCES staff(id),
  work_date   DATE NOT NULL,
  start_time  TIME NOT NULL,
  end_time    TIME NOT NULL,
  position    VARCHAR(50)
);

CREATE TABLE time_clock (
  id          BIGINT PK,
  staff_id    BIGINT NOT NULL REFERENCES staff(id),
  store_id    BIGINT NOT NULL REFERENCES store(id),
  work_date   DATE NOT NULL,
  clock_in_at TIMESTAMPTZ,
  clock_out_at TIMESTAMPTZ,
  is_corrected BOOLEAN NOT NULL DEFAULT false,
  corrected_by VARCHAR(255),
  correction_note VARCHAR(500)
);

```

4.10 監査・イベント・通知（基盤）

```

CREATE TABLE audit_log (
  id          BIGINT PK,
  company_code VARCHAR(20) NOT NULL, -- 非正規化コピー (FK制約なし。 §3.1)
  store_id    BIGINT REFERENCES store(id),
  actor       VARCHAR(255) NOT NULL,
  action      VARCHAR(50) NOT NULL,
  target_type VARCHAR(50) NOT NULL,
  target_id   BIGINT,
  before_summary TEXT,
  after_summary TEXT,
  ip          VARCHAR(45),
  device      VARCHAR(255),
  occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

```

) PARTITION BY RANGE (occurred_at);
CREATE INDEX ix_audit_log_company_occurred ON audit_log(company_code, occurred_at);

CREATE TABLE domain_event (
  id          BIGINT PK,
  company_code VARCHAR(20) NOT NULL, -- 非正規化コピー (FK制約なし。 §3.1)
  store_id    BIGINT REFERENCES store(id),
  aggregate_type VARCHAR(30) NOT NULL
    CHECK (aggregate_type IN ('TABLE_SESSION', 'ORDER_LINE', 'CHECK', 'PAYMENT', 'DAILY_CLOSE')),
  aggregate_id BIGINT NOT NULL,
  event_type   VARCHAR(50) NOT NULL,
  payload      JSONB NOT NULL,
  occurred_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  actor        VARCHAR(255) NOT NULL
) PARTITION BY RANGE (occurred_at);
CREATE INDEX ix_domain_event_aggregate ON domain_event(aggregate_type, aggregate_id);
CREATE INDEX ix_domain_event_company_occurred ON domain_event(company_code, occurred_at);

CREATE TABLE outbound_message (
  id          BIGINT PK,
  store_id    BIGINT NOT NULL REFERENCES store(id),
  type        VARCHAR(20) NOT NULL CHECK (type IN ('EMAIL', 'SMS')),
  "to"        VARCHAR(255) NOT NULL,
  template    VARCHAR(100) NOT NULL,
  status      VARCHAR(20) NOT NULL DEFAULT 'QUEUED'
    CHECK (status IN ('QUEUED', 'SENT', 'FAILED')),
  sent_at     TIMESTAMPTZ,
  error       VARCHAR(500)
);

```

`audit_log` と `domain_event` は §8 の保持方針に基づき、`occurred_at` の月次レンジパーティション（`pg_partman` で自動管理）とする。

5. インデックス方針

- 全テーブルの外部キー列には既定でB-treeインデックスを張る（上記DDLで明示していないFK列も同様）。
- 高頻度の一覧・検索クエリに対応する複合インデックスを上記DDL中にコメント付きで例示した（`table_session(store_id, status)` = 卓一覧、`guest_check(store_id, business_date)` = 日報集計等）。
- `dining_table.qr_token` と `mobile_order_session.qr_token` は読み取り頻度が高いため UNIQUE インデックス必須（既にDDLに含む）。
- `domain_event` / `audit_log` はパーティションキー（`occurred_at`）と `company_code` の複合インデックスを基本とし、詳細な絞り込みは分析基盤側（フェーズ2）で別途最適化する。

6. 実装方式の詳細

6.1 認証・認可

- ログインは `company_code + email + password` を入力とする（既存踏襲）。`company_code` は `users` の列ではなく `company` を検索して `company_id` に解決し、`company_id + email` で `users` を照合する（§3.1）。成功時に JWT（アクセストークン15分 / リフレッシュトークン14日）を発行し、クレームに `company_id`、`company_code`（解決結果。 `company_code` 列を持つ業務テーブルのテナントフィルタ用）、`user_id`、`store_id`（nullable）、`role` を含める。
- モバイルオーダーは未ログインのため JWT を発行しない。代わりに `mobile_order_session` の `qr_token` を署名付き短命トークン（JWTではなく単純なランダム文字列＋サーバ側セッション参照）としてクライアントの `sessionStorage` に保持し、リクエストヘッダで送る。

6.2 API設計方針

- REST + OpenAPI（springdoc-openapi で自動生成）。ベースパスは /api/v1。
- 認証系以外は原則 /api/v1/stores/{storeId}/... の配下に置き、storeId は必ずテナントコンテキストと照合する（他店舗IDを指定してもテナント外なら404を返す。存在有無を漏らさないため403ではなく404）。
- 一覧系はカーソルベースページング（?cursor=...&limit=...）を既定とする（created_at, id の複合キー）。
- エラーレスポンスは既存 ErrorResponse / ErrorItem を継承し、code（アプリ定義のエラーコード）、message、details[] を返す統一フォーマットとする。
- リアルタイム卓状況・モバイルオーダー受理通知は、フェーズ1はポーリング（5秒間隔）で開始し、WebSocket/SSE 化はフェーズ2の性能検証後に判断する（01 5.2 の想定を実装順序として確定）。

6.3 主要APIエンドポイント一覧（抜粋・フェーズ1）

リソース	メソッド・パス	対応FR
認証	POST /api/v1/auth/login、POST /api/v1/auth/refresh、POST /api/v1/auth/password-reset	FR-A01, A04
招待	POST /api/v1/stores/{storeId}/invitations、POST /api/v1/invitations/{token}/accept	FR-A03
店舗設定	GET/PUT /api/v1/stores/{storeId}/settings、.../tables、.../payment-methods、.../business-days	FR-B01～B09
予約	GET/POST /api/v1/stores/{storeId}/reservations、PATCH .../{id}、POST /api/v1/public/stores/{storeCode}/reservations（Web予約・認証不要）	FR-C01～C09
メニュー	GET/POST/PUT /api/v1/stores/{storeId}/menu-items、.../menu-categories	FR-D01～D05
卓・注文	POST /api/v1/stores/{storeId}/table-sessions、POST .../{id}/orders、PATCH .../order-lines/{id}	FR-E01～E07
モバイルオーダー	GET /api/v1/mobile/{qrToken}/menu、POST /api/v1/mobile/{qrToken}/orders、GET /api/v1/mobile/{qrToken}/orders	FR-F01～F11
会計	POST /api/v1/table-sessions/{id}/checks、POST .../checks/{id}/payments、POST .../checks/{id}/finalize、POST .../checks/{id}/refunds	FR-G01～G12
日次締め	POST /api/v1/stores/{storeId}/daily-closes、GET .../sales-daily-reports	FR-H01～H05
シフト・勤怠	GET/POST /api/v1/stores/{storeId}/staff、.../shift-requests、.../shift-schedules、POST .../time-clocks	FR-I01～I06
監査ログ	GET /api/v1/companies/{companyCode}/audit-logs	FR-J04
Webhook	POST /api/v1/webhooks/paypay、POST /api/v1/webhooks/credit-card	FR-G06, G07

6.4 税計算・端数処理

- 消費税額は guest_check_tax_line の税区分（STANDARD_10 / REDUCED_8）ごとに、対象額合計に税率を乗じたうえで**1回だけ**端数処理する（インボイス制度が要求する「1請求書につき税率ごとに1回」に対応）。端数処理方式は切り捨て（store_setting.tax_rounding の既定値 FLOOR。将来 CEIL / ROUND へ店舗ごとに変更できる余地は残すが、フェーズ1のUIでは変更不可＝固定運用とする）。
- 明細単位では税額を計算・表示しない（合計金額の突合ズレを避けるため）。

- 現金精算等で生じる1円未満の調整は `guest_check_discount(type='ROUNDING')` として明示的に記録する。

6.5 帳票（レシート／領収書）生成

- サーバサイドで `openhtmltopdf`（`org.xhtmlrenderer:openhtmltopdf-pdfbox`）を用い、Thymeleafで組んだHTMLテンプレートから直接PDFバイト列を生成する（外部プロセス起動なし。コンテナ化・スケールが容易なため `wkhtmltopdf` のような外部バイナリ依存は避ける）。
- 生成したPDFはオブジェクトストレージ（本番のホスティング確定後に選定。フェーズ1開発中はローカルディスクまたはMinIOで代替）に保存し、`receipt.pdf_ref` にキーを保持する。

7. 決済連携の詳細

7.1 決済手段プラグイン方式

各決済手段を共通インターフェースの実装として扱う。

```
public interface PaymentGateway {
    PaymentMethodType methodType();
    PaymentResult charge(ChargeRequest request);           // 同期実行可能な手段のみ
    PaymentResult reconcile(String externalTxnId);         // Webhook/照会での状態確定
}
```

実装	方式	備考
CashPaymentGateway	アプリ内完結（外部呼び出しなし）	預り金・釣り銭計算のみ
PayPayPaymentGateway	PayPay 加盟店API（動的QR）＋Webhook	連携失敗時は <code>FAILED</code> を記録し、手入力の <code>SUCCESS</code> で消し込み（FR-G06）
CreditCardPaymentGateway	決済代行SDK（第一候補 Square）＋Webhook	サービス最終確定は 02.11.1 の経営判断待ち
RakutenPayManualGateway	常に <code>is_manual_entry=true</code> で即時 <code>SUCCESS</code>	システム連携なし（FR-G07b）

7.2 会計・決済フロー

- スタッフが決済手段を選択し `payment` を `PENDING`（外部連携あり）または即時 `SUCCESS`（現金・楽天ペイ）で作成。
- 外部連携ありの手段は決済端末／SDKを起動し、結果をWebhookまたは同期レスポンスで受け取る。
- $\sum \text{payment}(\text{status}=\text{SUCCESS}).\text{amount_jpy} = \text{check}.\text{total_jpy}$ になった時点で `guest_check` を `FINALIZED` にする（03.4.6 の不変条件どおり）。

7.3 Webhookの冪等性・リトライ

- Webhook受信エンドポイントは `external_txn_id` を必須とし、`payment` テーブルの部分UNIQUEインデックス（§4.7）で二重処理を防ぐ。再送されたWebhookは既存レコードの状態確認のみ行い、重複した金銭計上をしない。
- Webhook側の失敗（5xx応答）はPayPay/決済代行側の標準リトライに委ね、アプリ側からの追加リトライは行わない。
- 決済代行のAPIキー等（`payment_method_config.credential_enc`）はアプリ層でAES-256-GCM暗号化して保存し、復号は決済リクエスト時のみメモリ上で行う。

7.4 モバイルオーダーの `qr_token` 運用

- `dining_table.qr_token`：卓に印刷されたQRに埋め込む固定トークン。店舗設定画面で手動再発行可能（紛失・いたずら対策として、再発行時は旧トークンを即時無効化）。
- 読み取り時、対応する `table_session` が `OPEN` であれば `mobile_order_session` を新規発行し、そのセッ

ション固有の短命 `qr_token`（実体はランダム文字列、有効期限は卓クローズ or 発行から12時間のいずれか早い方）をクライアントへ返す。

- `table_session` が `CLOSED` になると、紐づく全 `mobile_order_session` を `EXPIRED` にする（03 4.2のとおり）。

8. `domain_event` の実装方式

- **粒度**：03 2.8 で定義済みの `aggregate_type`（`TABLE_SESSION` / `ORDER_LINE` / `CHECK` / `PAYMENT` / `DAILY_CLOSE`）ごとの主要な状態変化のみを記録する。カラム単位の変更履歴（audit的な差分）は `domain_event` の対象外とし、必要なら `audit_log` 側で扱う。
- **書き込み方式**：Transactional Outbox パターンを採用する。業務トランザクションのコミットと同一トランザクション内で `domain_event` へ INSERT する（別テーブルへの二相コミットは行わない）。フェーズ1では `domain_event` を分析基盤へ配信する仕組み（Kafka等へのCDC）は持たず、テーブルへの蓄積のみ行う。フェーズ2でAIオーケストレーション層・分析基盤を追加する際に、このテーブルをポーリングまたはCDC（Debezium等）で配信する構成に拡張する。
- **保持・アーカイブ方針**：
 - 直近13か月分は `domain_event` 本体（月次パーティション）でオンライン参照可能に保持する。
 - 13か月を超えたパーティションは、コールドストレージ（本番ホスティング確定後にS3互換ストレージへのエクスポートを想定）へJSONL形式でエクスポートしたうえでパーティションをデタッチする。
 - 保持期間は § 11 のデータ保持方針（10年）に従う。10年経過したパーティションはコールドストレージ側でも削除する。

9. オフライン同期の競合解決規則

- **対象範囲**：オフライン継続を許すのは「スタッフ端末での注文入力（新規追加のみ）」に限定する（02 NFR-05/06 のとおり、会計・決済・モバイルオーダーはオンライン必須）。
- **クライアント一時ID（端末ID+連番方式）**：
 - **端末IDの採番元**：サーバがデバイス登録時に払い出す（スタッフ端末を店舗へ登録する操作の中で発番する。クライアントの自己申告は認めない）。形式は「`store` の短縮コード+店内連番」（例 `S12-07`）で、`staff_device(store_id, device_code)` を `UNIQUE` とする（§ 4.6）。
 - **一時IDの構成**：オフライン中にクライアントが生成する `customer_order`・`order_line`・`order_line_option` の各レコードには、`端末ID` + "-" + 端末内で単調増加する連番（例 `S12-07-000123`）を `client_ref_id`（`VARCHAR(40)`）としてレコードごとに持たせる。連番は卓・日付・親レコードでリセットせず、端末内でグローバルに増加させる（1端末=1本の採番列）。
 - **冪等性（サブツリー全体）**：オンライン復帰時、サーバは各テーブルの `UNIQUE` 制約——`customer_order(table_session_id, client_ref_id)`、`order_line(table_session_id, client_ref_id)`、`order_line_option(order_line_id, client_ref_id)`（いずれも § 4.6）——により、**親だけでなく 明細・オプションの各レコード単位で再送を無視する**。既に `id`（`bigint`）を発行済みなら、サブツリー全レコードの `client_ref_id` → `id` 対応をレスポンスで返す（冪等な再送信= どの階層も二重登録しない）。あわせて `staff_device.last_accepted_seq` を更新し、それ以下の連番を持つリクエストは再送とみなして拒否する。`client_ref_id` は `NULL` 可（オンラインでサーバが直接作成したレコードには付与しない）。`UNIQUE` 制約は `NULL` どうしを重複と見なさないため、オンライン作成分は制約の対象外となる。
 - **端末の再セットアップ**：端末を初期化した場合は `staff_device.status` を `RETIRED` にして新しい端末IDを再発番する。旧端末IDは再利用しない（連番が1に戻って過去レコードと衝突するのを防ぐ）。
- **同期ペイロードの形式（フラット+一時IDのFK）**：
 - サブツリー（親 `customer_order` + `order_line` × N + `order_line_option` × M）は、**入れ子にせずフ**

ラットなレコード配列として送る。各レコードは `type` と自身の `client_ref_id` を持ち、親へのFK列 (`order_id` / `order_line_id`) には親レコードの `client_ref_id` (一時ID文字列) を入れる。サーバは受信レコードを `client_ref_id` で索引化し、FK 値が索引に一致する辺で依存グラフを組んでトポロジカルソートしてから、親→子の順に INSERT しつつ一時ID→確定IDへ差し替える。

- **remake_of_line_id と remake_of_line_ref は分けて持つ。**同期済み明細を指す場合は `remake_of_line_id` (確定 bigint) に入れてそのまま使う。同一バッチ内の未同期明細を指す場合は `remake_of_line_ref` (一時ID文字列) に入れ、サーバが対応表で確定IDへ解決して `order_line.remake_of_line_id` に格納する (`remake_of_line_ref` はペイロード専用フィールドで、テーブル列としては保持しない)。両方が同時に入ることはない。
 - FK 値が一時ID文字列か確定 bigint かは値の形で判別する (一時IDは 端末コード・連番 形式の文字列、確定IDは数値)。`order_id` / `order_line_id` は原則つねに一時ID (同一バッチで新規作成した親を指すため)。
- **競合の設計上の回避：**数量変更・取消はオンライン必須の操作としたため、複数端末が同一 `order_line` を同時に更新する競合はそもそも発生しない (追加のみのオフライン操作は他の追加と独立しており、マージ不要)。これにより「後勝ち／マージ／要確認」のような複雑な競合解決規則の実装を不要にする。
 - **オフラインで許可するのは新規レコードの作成のみ (既存行の更新は不可)：**すでにサーバへ永続化済みの行——オンラインで作成され `client_ref_id` を持たない `order_line` / `customer_order` 等——への更新は、`serve_status` の `PENDING` → `SERVED` を含め**オフラインでは一切行わない**。`updated_at` / 専用の `version` 列を初期表示時にクライアント保持し復帰時にサーバ現在値と突き合わせる**楽観的ロックでの解禁は採用しない**：長時間オフラインでは版の不一致が高頻度で発生し、エラー後の手動照合がかえって増えるため (競合検知の手段としては有効なので、複数端末が同一卓を同時操作する通常運用の保護として オンライン更新経路側に `version` 列で導入することは別途検討してよい)。オフライン中に既存行を更新したい操作は、アプリがオフライン時に抑止するか、意図をローカルキューへ退避してオンライン復帰後に通常の オンライン更新として再生する。
 - オフラインで**新規作成された明細は**、初回同期ペイロードで端末上の `serve_status` / `served_at` を **INSERT 時の初期状態**として持ち込んでよい (下記「`kitchen_ticket` 抑止＝案1」で確定)。既存行への UPDATE ではないため上記の禁止対象には当たらない。
 - **`kitchen_ticket` 抑止 (提供済みオフライン明細で KDS を鳴らさない) ＝案1 (確定)：**オフライン作成の `order_line` は同期ペイロードに端末上の `serve_status` (提供済みなら `served_at` も) を初期状態として含める。サーバは受信時、`serve_status` が `PENDING` 以外の明細については**新規 `kitchen_ticket` を KDS に鳴らさない**。`serve_status = PENDING` のオフライン明細は従来どおり `kitchen_ticket` を発行し KDS に表示する (表示順は `registered_at` 基準)。サーバの操作は INSERT 1回のみで既存行 UPDATE は発生しないため、「オフラインは新規追加のみ・既存行の更新は不可」と矛盾しない。
 - **粒度＝G1 (確定)：**`kitchen_ticket` は従来どおり `customer_order` 単位で1件発行する (行単位に分割しない)。オーダー内の**全明細が `SERVED` の場合のみ**、そのチケットを **`status = DONE` で作成し KDS には表示しない** (レコードは監査・スループット分析用に残す)。**1つでも `PENDING` を含むオーダー**はチケットを通常どおり KDS に出し、調理ビューの表示明細を `serve_status = PENDING` のものだけに絞る (`SERVED` 明細は KDS 表示クエリのフィルタで除外。スキーマ変更なし)。
 - 細目は未決 (03 7章3)：「鳴らさない」をアラート抑止のみ (ミュートの照合レーンには出す) とするか 完全非表示 とするか。抑止 (`DONE`) したチケットを KDS の滞留時間・スループット指標から除外するか。
 - **`CLOSED` セッション / `FINALIZED` check への着地 (フェーズ1方針)：**更新競合は無くても、端末Aがオフラインで明細を溜めている間に別のオンライン端末Bが同じ卓の会計を確定し `table_session` が `CLOSED` になる、という時間差は残る。復帰後の端末Aの同期が `CLOSED` セッション (または `FINALIZED` な check) に着地した場合、1段目のサブツリー検証で落とし、親を `REJECTED` (`error.code = SESSION_CLOSED`) ・配下を `SKIPPED` で返す (従来どおり)。**フェーズ1では、拒否された明細を業務的に回収しない——** `FINALIZED` check は不変のため明細追加・補助会計・refund による代金回収は行わず、当該明細は**回収不能 (廃棄ロス / サービス提供分)**として `domain_event` (`aggregate_type = ORDER_LINE` の監査用イベント) に記録するのみとする。スタッフ端末にはエラー表示とエスカレーション 通知を出

し、店舗側は物理的な提供実績と突き合わせて棚卸し・ロス計上で処理する。代金回収経路（追加請求・翌営業日補正）の整備はフェーズ2以降とする。なお、端末のオフライン許容時間の上限（超過時に新規入力を止めるか否か）は未決とする（03 7章3）。

- **スナップショットの鮮度（オフライン中は端末保持のメニューで確定）**：オフライン作成明細の `item_name_snap` / `unit_price_snap_jpy` / `tax_category_snap`、および `order_line_option` の `option_name_snap` / `price_delta_snap_jpy` は、注文時点で端末ローカルのメニューキャッシュから採ったスナップショット値を採用する。オンライン復帰時、サーバは現行の `menu_item` / オプションマスタで **価格・名称・税区分を再計算しない**——オフライン中にマスタが変わっていても端末が持っていた値で確定させる。`order_line` は元々注文時点のスナップショット列を持つ設計であり、この決定は「端末のキャッシュが古くてもよく、サーバは復帰時に補正しない」ことを明文化するもの。
 - 対象商品がオフライン中に `SOLD_OUT` / `SUSPENDED` / `is_active = false` になっていた場合に、明細単位 検証（本節「部分失敗時の扱い」2.）で `SOLD_OUT` / `ITEM_SUSPENDED` / `ITEM_INACTIVE` として却下するか、提供済み前提でそのまま通すかは未決（03 7章3、提供済みオフライン明細クラスタ）。
- **オフライン中の端末時計とタイムスタンプ（端末時刻+スキュー補正）**：オフライン作成レコードの時刻は、端末時計の値をサーバ側でスキュー補正して確定する。
 - `client_sent_at`：同期ペイロードのエンベロープに `client_sent_at`（バッチ送信時点の端末時計値）を追加する。サーバは `offset = server_received_at - client_sent_at` を算出し、バッチ内の各オフライン時刻——`order_line.registered_at`、`customer_order.submitted_at`、対応する `domain_event.occurred_at`、およびオフライン明細が自前の `served_at` を持ち込む案（本節「新規レコードの作成のみ」の未決サブ項目）を採る場合の `order_line.served_at`——に**一律加算**する。「オフライン継続中はオフセットがおおむね一定」（時計が一定量ズレているだけ）を前提として許容する。
 - **補正值の格納と低信頼フラグ（案の基本）**：補正後の時刻は**そのまま格納し、低信頼フラグを立てる**（分析・集計側がこのフラグで除外できる）。フラグは永続化する想定で、`order_line` / `customer_order` に真偽値列を持たせる方向（列名・配置は § 4.6 DDL で最終化）。同期レスポンスの `server_fields` でも当該時刻とフラグをエコーバックする。
 - **時系列破綻時のみサーバ時刻へ置換**：補正後の値が時系列的にあり得ない場合（対象 `table_session` の開始前、`server_received_at` より未来、など）は、その時刻だけ `server_received_at`（またはバッチ受信時刻）で置換する。置換したレコードにも低信頼フラグを立てる。置換は「相対間隔を保って全体をずらす」のではなく該当時刻単位で行う。
 - **offset の許容上限 X**：`|offset|` が X を超えたら定常ドリフトの範囲外（時計がオフライン中に変更された等）とみなす。上限超過時も上記（低信頼フラグ+時系列破綻分のみ置換）と同じ扱いとし、超過 そのものを理由にレコードを拒否はしない。X は**運用設定値**として持つ——**システム全体の単一既定値**とし、店舗別オーバーライドはフェーズ2以降。端末のクロックドリフトは機種・OS 特性であり店舗業務に依存しないため `store` 単位では持たない。格納方式（汎用設定テーブルの新設か、アプリ設定〔環境変数/設定ファイル〕か）と、既定値・安全に設定できるレンジは実機のドリフト実測に基づき**実装スパイクで確定**する。
 - **business_date の帰属（確定）**：`order_line` に `business_date` DATE NOT NULL 列を追加し（A2）、登録時に**補正後 registered_at + 店舗の営業日境界（store_business_day）から算出**する（B1）。`guest_check.business_date` は従来どおり精算日を保持し、明細の「注文された営業日」とは別に持つ。ただし `time_low_confidence = true` の明細は `registered_at` を信用せず、`business_date` を紐づく `guest_check.business_date`（B2）で決める。
 - **B1 の算出先が締め済み営業日だった場合=D2（確定）**：端末が `daily_close` をまたいでオフラインだった結果、補正後 `registered_at` が `daily_close = CLOSED` の営業日を指す場合は、当該明細を**拒否せず受け入れ**、`business_date` を締め済み日ではなく**同期処理時点でオープン中の営業日**に設定して当日計上する（D2）。補正後 `registered_at` は実際の注文時刻として列にそのまま残す。明細には「前営業日からの遅延計上」を示す理由コード/フラグを立てる（列名、および `sales_daily_report` での前日遅延計上の表示方法は実装スパイクで確定）。締め済みの `daily_close` 側には遅延計上ありの通知・フラグは**行わない**（CLOSED の不変性を維持）。
 - **KDS 表示順の基準時刻= registered_at（確定）**：KDS のチケット表示順は、`kitchen_ticket` の生

成時刻（printed_at＝オフライン分は同期到着時刻）ではなく、**注文明細の registered_at（オフラインは補正後）**を基準にする。kitchen_ticket は customer_order 単位なので、ソートキーはその注文の入力時刻（customer_order.submitted_at、＝配下 order_line.registered_at の最小値。両者は入力時に同一補正で確定する）とする。kitchen_ticket ヘソート用時刻を非正規化コピーするかはクエリ実装の詳細。time_low_confidence の明細は表示位置がずれ得るが、KDS は一時的表示で不変データを持たないため許容する。どの明細をそもそも KDS に出すか（提供済みオフライン分の抑止）は別項目「kitchen_ticket 抑止」で決める。

- **未決**：端末の生時刻を別列で保持するか、復帰時に端末時計をサーバへ同期するか、time_low_confidence 列の最終的な名称・配置、および D2 の遅延計上フラグの列名（03 7章 3）。
- **復旧処理**：ネットワーク復帰時、クライアントはキューに溜めた未送信レコードを、親子1組（サブツリー）を1トランザクションとして上記フラット形式で送信順に再生する。サーバ側の処理順は受信順でよい（同一卓内の注文は追記のみで順序整合性への影響がないため）。
- **依存順の解決はアプリ層のトポロジカルソートで行う**：サーバはサブツリー受信後、client_ref_id の索引と FK 値（order_id／order_line_id／remake_of_line_ref）からバッチ内の依存グラフを構築し、アプリ層でトポロジカルソートしてから、参照先が先に来る順で INSERT する。DB の遅延制約（DEFERRABLE INITIALLY DEFERRED）は採用しない。理由：(1) サーバは一時ID→確定IDの差し替えのためにどのみち依存グラフを組む必要があり、ソートの追加コストがほぼ無い、(2) FK 制約を通常の IMMEDIATE チェックのまま保てるため、不正な参照は問題の INSERT 時点で即座に検出でき、COMMIT まで遅れない、
- 3. FK に DEFERRABLE を付ける DDL 変更が不要で、他テーブルと制約の扱いを揃えられる。
- **レスポンス形式**：バッチ全体のエンベロープと、送信レコード全件に対応する records[] を返す。

```
{
  "server_received_at": "2026-09-08T12:34:56+09:00",
  "last_accepted_seq": 123,
  "records": [
    { "type": "customer_order", "client_ref_id": "S12-07-000123", "id": 5001, "status": "INSERTED" },
    { "type": "order_line", "client_ref_id": "S12-07-000124", "id": 88010, "status": "INSERTED",
      "server_fields": { "registered_at": "2026-09-08T11:02:00+09:00" } },
    { "type": "order_line_option", "client_ref_id": "S12-07-000125", "id": 90210, "status": "INSERTED" }
  ]
}
```

- エンベロープ：server_received_at（サーバ受信時刻）、last_accepted_seq（更新後の staff_device.last_accepted_seq のエコーバック。クライアントが確定済み位置を知る）。
- records[] の各要素：type（customer_order／order_line／order_line_option）、client_ref_id（一時ID）、id（確定 bigint。status が INSERTED／DUPLICATE のとき必須、REJECTED のときは省略）、status、任意の server_fields（サーバが上書き・導出した値。どの項目を上書きするかは端末時計の信頼性の未決事項に従う）、REJECTED 時の error（{ code, message, field? }）。
- status：INSERTED（新規登録）／DUPLICATE（冪等スキップ＝再送。既発行の同じ id を返す）／REJECTED（業務検証で却下。error を持つ）／SKIPPED（上位レコードの失敗により検証・挿入を見送り。error は持たず、原因となった祖先の client_ref_id を skipped_due_to で示す）。
- error.code の初期セット（文字列。§ 4.1 の enum 方針に準拠しアプリ側で定数管理）：
SOLD_OUT／ITEM_SUSPENDED／ITEM_INACTIVE／SESSION_CLOSED／INVALID_QUANTITY／OPTION_INACTIVE
／REMAKE_TARGET_NOT_FOUND／PERMISSION_DENIED。
- **網羅性**：送信した全レコードについて必ず1要素を返す。クライアントはこれで全件の張り替え完了を検証する。**再送安定性**：同じサブツリーを再送しても各 client_ref_id に対して同じ id を返す（status は DUPLICATE になりうる）。
- レスポンスもリクエストと同じくフラット構造（入れ子にしない）。
- **部分失敗時の扱い（ハイブリッド）**：サーバ検証は2段階で行う。
 1. **サブツリー前提の検証（all-or-nothing）**：親 customer_order の妥当性、対象 table_session が注

文受付可能な状態か、依存グラフが解決できるか（構造整合性）。ここで落ちたら**サブツリー全体をロールバックし、何もコミットしない**。親を REJECTED（該当 error.code）、配下の全 order_line / order_line_option を SKIPPED（skipped_due_to = 親の client_ref_id）で返す。

2. **個々の明細の業務検証（部分コミット）**：order_line / order_line_option を1件ずつ検証する。通ったレコードと親はコミットする。落ちた order_line は REJECTED、その配下の order_line_option は SKIPPED（skipped_due_to = 当該明細の client_ref_id）で返す。他の正常な明細のコミットには影響しない。
- **リトライ可否**：レスポンスに明示フィールドは置かない。error.code 付きの REJECTED は**永続的失敗**とみなし、クライアントは自動再送せず、当該明細をエラー状態にしてスタッフヘスケーレーションする。**一時的失敗はトランスポート層で判定する**——同期API呼び出しが 5xx / タイムアウト / ネットワークエラーで 返った場合は、バッチ全体を指数バックオフで再送する（サブツリーは冪等なので安全）。将来 after_delay 相当（一定時間後に自動再試行すべき業務失敗）が必要になった時点で、retryable + retry_after を追加する。
 - 却下された明細に対応する料理がオフライン中にすでに提供済みだった場合の**監査／廃棄ロス連携**は、上記「CLOSED セッション / FINALIZED check への着地」と同じ扱い——回収不能として domain_event に記録し、棚卸し・ロス計上で処理する——をフェーズ1の既定とする。部分失敗バッチの HTTP ステータス 規約は実装スパイクで確定する（03 7章3）。

- **クライアント側の一時ID↔確定ID対応表の保持期間**：クライアントは、あるレコードの client_ref_id → 確定id 対応を、そのレコードが属する table_session が CLOSED になるまで ローカル（IndexedDB 等）に保持し、CLOSED を検知した時点で当該セッション分のエントリを破棄する（CLOSED 後はその明細に対する操作が発生しないため）。対応表は各ローカルレコードとは別の 索引として持つ。アプリ再インストールをまたいだ保持は不要（端末初期化時は staff_device が RETIRED になり端末IDを再発番するため、旧一時IDは以後使われない）。バックストップとして、CLOSED を長期間観測できないセッションのエントリは固定TTL（例：72時間）で破棄する。サーバ側は customer_order / order_line / order_line_option の client_ref_id 列を § 10 の 保持期間まで永続保持するため、これはクライアント端末のローカルコピーの寿命のみを定める規定。

10. データ保持・アーカイブ方針（確定値）

02 2.5 および DAT-04 / NFR-13 の「仮置き」値を、以下のとおり確定する（02 11.2 の法務確認は引き続き並行実施し、確定後に本章を必要なら改訂する）。

データ区分	対象テーブル	保持期間	根拠・備考
取引・帳簿系	order_line, guest_check*, payment, refund, daily_close, sales_daily_report*, receipt	10年	法人税法上の帳簿書類保存期間（原則7年、欠損金繰越等で最長10年）に安全側で合わせる。
ドメインイベント	domain_event	オンライン13か月＋コールド10年	§ 8 のとおり。分析・AI基盤の直近データ参照性能と長期保持を両立。
監査ログ	audit_log	5年	02 の「重要操作5年」に統一し、区分運用（1年/5年の出し分け）による実装・運用の複雑化を避ける。
予約・個人情報	reservation（guest_* 列）, user, staff	退会時に削除。通常運用中はテナント契約継続中は保持	DAT-05。テナント退会時のデータ削除手順は § 11 で別途定義。
シフト・勤怠	shift_*, time_clock	労働基準法の記録保存義務に合わせ 5年 （賃金台帳等の保存期間の目安）	CON-06

11. 退会・データ削除

- テナント（`company`）退会時、`contract_status` を `CANCELLED` にしたうえで、90日間は復元可能な 猶予期間としてデータを保持する。猶予期間経過後、個人情報を含むテーブル（`user`、`reservation` の `guest_*` 列、`staff`、`time_clock` 等）を対象にPIIを匿名化（氏名・連絡先をハッシュ化した仮名に置換）し、取引金額等の集計に必要な行自体は § 10 の保持期間まで残す。
- エクスポート（DAT-07）は退会前に必ず案内し、CSVダウンロードの機会を提供する。

12. Mermaid図のPDFレンダリング方針（確定・実運用済み）

03 冒頭で「PDF化ではコードのまま出力されるため、プリレンダリングが必要」としていた論点を、2026-09-05 に本リポジトリの `docs/pdf/*.pdf` 生成で以下のとおり確定・実運用した。

- `docs/*.md` 中の `mermaid` フェンス（````mermaid`）を、`npx --yes @mermaid-js/mermaid-cli@11`（`mmdc`）で図ごとにPNG化する（背景白、フォントは `Noto Sans CJK JP`）。
- `mermaid-cli` はヘッドレスChromeを介して描画するため、実行環境に `libnss3` / `libnspr4` / `libasound2t64` が必要（WSL2 の最小構成には無く、`sudo apt-get install` が別途必要だった）。
- PNG化した図をMarkdownの一時コピーに埋め込み、`pandoc --pdf-engine=wkhtmltopdf` でPDF化する。日本語表示には `Noto Sans CJK JP`（本文）／`Noto Sans Mono CJK JP`（コード）を指定したCSSを用いる。
- この手順はドキュメントPDF化専用であり、§ 6.5 の帳票（レシート／領収書）生成では採用しない（本番サーバに Chrome／Node.js を同梱したくないため）。

13. 実行環境・CI/CD（01 5.6 を確定）

- 本番実行環境：コンテナ（`Docker`）。ホスティング先（クラウド事業者）は未確定（02 11.1 の経営判断待ち）。
- マイグレーション：Flyway。デプロイパイプラインで `flyway migrate` をアプリ起動前に実行する。
- CI：GitHub Actions で `mvn verify`（バックエンド：単体テスト＋ArchUnitのテナント境界検査）、`npm run build && npm test`（フロントエンド）を実行する。
- Hibernate の `ddl-auto` は `validate` 固定（§ 2）。スキーマ変更は必ずFlywayマイグレーション経由。

14. 次のステップ

- 本書のレビュー（DDLの過不足、`customer_order` / `guest_check` への物理テーブル名変更の影響確認）。
- Flyway `V1__init_schema.sql` として本書 § 4 のDDLを実装し、既存 `users` テーブルへのマイグレーション（§ 4.3）を検証環境で先行実施する。
- 02 11.1／11.2 に残る経営判断・法務確認（決済代行の最終選定、保持期間の最終法務確認）と並行して、フェーズ1のバックログ化（エンティティ単位のCRUD・状態遷移をまたぐユースケースのストーリー分解）を進める。